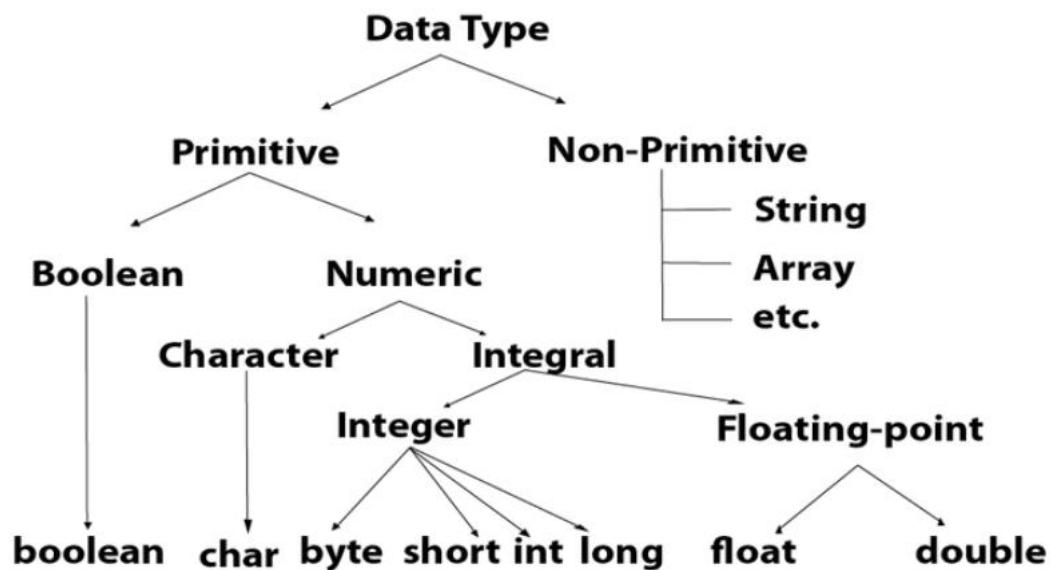


Java Data Types

Java data types define the type of data a variable can store in a program.



Data Type	Default Value	Default size	Example	Range
boolean	false	1 bit	Boolean one = false	true and false
char	'\u0000'	2 byte	char ch = 'A'	'\u0000' (or 0) to '\uffff' (or 65,535) (or) 0 to 65,536
byte	0	1 byte	byte a = 10	-128 to 127
Short	0	2 byte	short s = 10000	-32,768 to 32,767
int	0	4 byte	int a = 100000	-2,147,483,648 to 2,147,483,647
long	0L	8 byte	long a = 100000L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	0.0f	4 byte	float f1 = 234.5f	1.4e-045 to 3.4e+038
double	0.0d	8 byte	double d1 = 12.3	4.9e-324 to 1.8e+308

Primitive Data Types

Primitive data types are predefined by Java and are used to store simple values.

Data Type	Description	Example
byte	Small integer	byte age = 20;
short	Short integer	short year = 2026;
int	Integer	int marks = 95;
long	Large integer	long population = 8000000000L;
float	Single-precision decimal	float price = 99.5f;
double	Double-precision decimal	double pi = 3.14159;
char	Single Unicode character	char grade = 'A';
boolean	Logical value	boolean passed = true;

```
public class Main {  
    public static void main(String[] args) {  
        byte age = 25;  
        int salary = 50000;  
        System.out.println("Age = " + age);  
        System.out.println("Salary = " + salary);  
    }  
}
```

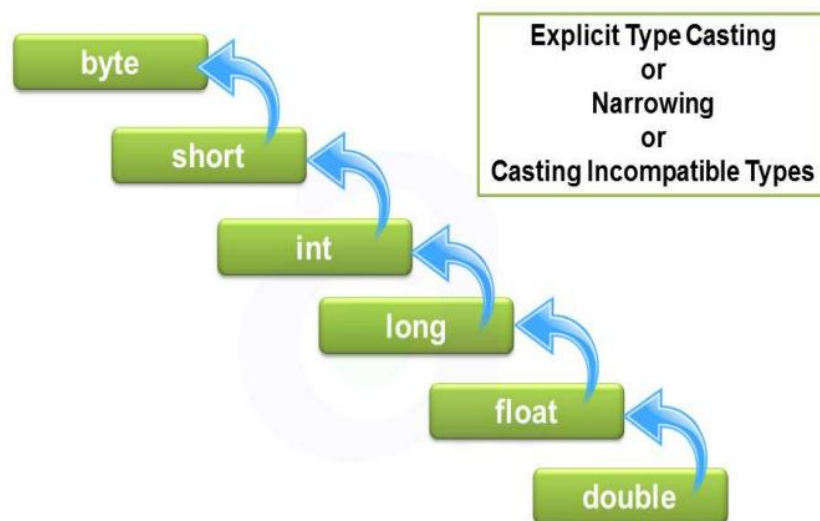
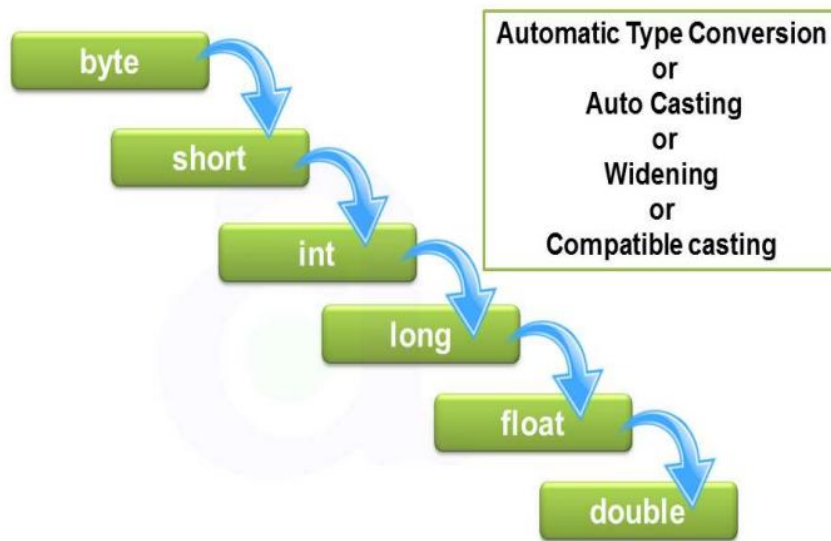
Age = 25
Salary = 50000

Overflow

```
public class Main {  
    public static void main(String[] args) {  
        byte b = 127;  
        b++;  
        System.out.println(b);  
    }  
}
```

-128

Type Conversion and Casting



Widening (Automatic)

```
public class Main {  
    public static void main(String[] args) {  
        byte b = 100;  
        int i = b; // Automatic conversion  
        System.out.println(i);  
    }  
}
```

100

Narrowing (Manual)

```
public class Main {  
    public static void main(String[] args) {  
        int i = 100;  
        byte b = (byte)i;  
        System.out.println(b);  
    }  
}
```

100

Arithmetic with byte: Arithmetic on byte, short, and char always produces an int.

```
public class Main {  
    public static void main(String[] args) {  
  
        byte a = 10;  
        byte b = 20;  
  
        // byte c = a + b; // Compilation Error (int + int)  
  
        int c = a + b;  
  
        //Type Promotion means Java automatically converts smaller data types to  
        //a larger data type before performing an operation.  
  
        System.out.println(c);  
    }  
}
```

30

Use Explicit Casting

```
public class Main {  
    public static void main(String[] args) {
```

```
byte a = 10;
byte b = 20;

byte c = (byte)(a + b);

System.out.println(c);
}
```

30

(OR)

```
public class Main {
    public static void main(String[] args) {

        byte a = 10;
        byte b = 20;

        int c = a + b;

        System.out.println(c);
    }
}
```

30

```
public class Main {
    public static void main(String[] args) {
        char ch1 = 'A';
        char ch2 = 'B';
        //char ch3 = ch1 + ch2; // Error
        int result = ch1 + ch2;

        System.out.println(result);
    }
}
```

131

Be Careful with Overflow

```
public class Main {  
    public static void main(String[] args) {  
  
        byte a = 100;  
        byte b = 100;  
  
        byte c = (byte)(a + b);  
  
        System.out.println(c);  
    }  
}
```

-56

```
// Demonstrate char data type.  
class CharDemo {  
    public static void main(String args[]) {  
        char ch1, ch2;  
        ch1 = 88; // code for X  
        ch2 = 'Y';  
        System.out.print("ch1 and ch2: ");  
        System.out.println(ch1 + " " + ch2);  
    }  
}  
  
output:  
ch1 and ch2: X Y
```

```
public class Main {  
    public static void main(String[] args) {  
        // Constant assignments (Valid)  
        byte b = 100;  
        short s = 30000;  
        char ch = 88;    // Unicode 88 = 'X'  
        int i = 50000;  
        long l = 100;  
  
        System.out.println("byte = " + b);  
    }  
}
```

```

System.out.println("short = " + s);
System.out.println("char = " + ch);
System.out.println("int = " + i);
System.out.println("long = " + l);

// Widening conversions (Valid)
byte b1 = 10;

short s1 = b1; // byte -> short
int i1 = s1;    // short -> int
long l1 = i1;   // int -> long

System.out.println("\nWidening Conversions:");
System.out.println("short = " + s1);
System.out.println("int = " + i1);
System.out.println("long = " + l1);

// Narrowing conversions
int x = 100;

// byte b2 = x;    // ERROR
// short s2 = x;   // ERROR
// char ch2 = x;   // ERROR

// Explicit casting
byte b2 = (byte) x;
short s2 = (short) x;
char ch2 = (char) x;

System.out.println("\nAfter Explicit Casting:");
System.out.println("byte = " + b2);
System.out.println("short = " + s2);
System.out.println("char = " + ch2); // 'd' because Unicode 100 = 'd'
}
}

```

```

public class Main {
    public static void main(String[] args) {

        // float initialization
        float f1 = 10;    // int -> float (automatic widening)
        float f2 = 10.5f; // Correct (float literal)
        float f3 = (float)10.5; // Correct (casting)
    }
}

```

```

// double initialization
double d1 = 10;    // int -> double (automatic widening)
double d2 = 10.5;  // Correct (double is default)
double d3 = 10.5f; // float -> double (automatic widening)

System.out.println("f1 = " + f1);
System.out.println("f2 = " + f2);
System.out.println("f3 = " + f3);

System.out.println("d1 = " + d1);
System.out.println("d2 = " + d2);
System.out.println("d3 = " + d3);

// Invalid Examples
// float f4 = 10.5; // Compilation Error
// float f5 = d2;   // Compilation Error (double -> float)
}
}

f1 = 10.0
f2 = 10.5
f3 = 10.5
d1 = 10.0
d2 = 10.5
d3 = 10.5

```

Boolean

```

public class Main {

    // Instance variable (default value)
    boolean defaultValue;

    public static void main(String[] args) {

        // Explicit initialization
        boolean b1 = true;
        boolean b2 = false;

        // Boolean from a comparison
        boolean b3 = (10 > 5);
        boolean b4 = (10 == 20);

        Main obj = new Main();
    }
}

```

```
        System.out.println("b1 = " + b1);
        System.out.println("b2 = " + b2);
        System.out.println("b3 = " + b3);
        System.out.println("b4 = " + b4);
        System.out.println("Default value = " + obj.defaultValue);
    }
}
```

b1 = true
b2 = false
b3 = true
b4 = false
Default value = false

```
public class Main {

    boolean defaultValue; //instance variable

    public static void main(String[] args) {

        Main obj = new Main();

        System.out.println(obj.defaultValue);
    }
}
```

false

```
public class Main {

    static boolean defaultValue;

    public static void main(String[] args) {

        System.out.println(defaultValue);
    }
}
```

False

Member	Belongs to	Access from main()
boolean defaultValue;	Object (instance)	☐ Need an object
static boolean defaultValue;	Class	☐ Directly accessible

Character Escape Sequences

```

public class EscapeExample
{
    public static void main(String args[])
    {
        String str = new String ("My name is \'abcd\'");
        System.out.println(str);

        str = "My Lab is \"JAVA LAB\"";
        System.out.println(str);

        str = "My work files are in D:\\Work Projects\\java";
        System.out.println(str);

        str = "First Line \nSecond Line";
        System.out.println(str);

        str = "Ravindra\b\b" + "College";
        System.out.println(str);

        str = "Ravindra \rCollege";
        System.out.println(str);

        str = "Ravindra \fCollege";
        System.out.println(str);
    }
}

```

```

C:\Users\RCEW LAB 1>java EscapeExample
My name is 'abcd'
My Lab is "JAVA LAB"
My work files are in D:\Work Projects\java
First Line
Second Line
RavindCollege
Collegea
Ravindra  College
C:\Users\RCEW LAB 1>

```

Variables

The variable is the basic unit of storage in a Java program.

- A variable is defined by the combination of an identifier, a type, and an optional initializer.

- In addition, all variables have a scope, which defines their visibility, and a lifetime

```
int a, b, c;           // declares three ints, a, b, and c.
int d = 3, e, f = 5;   // declares three more ints, initializing
                      // d and f.
byte z = 22;           // initializes z.
double pi = 3.14159;   // declares an approximation of pi.
char x = 'x';          // the variable x has the value 'x'.
```

```
// Demonstrate dynamic initialization.
```

```
class DynInit {
    public static void main(String args[]) {
        double a = 3.0, b = 4.0;
        // c is dynamically initialized
        double c = Math.sqrt(a * a + b * b);
        System.out.println("Hypotenuse is " + c);
    }
}
```

The Scope and Lifetime of Variables

```
// Demonstrate block scope.
class Scope {
    public static void main(String args[]) {
        int x; // known to all code within main
        x = 10;
        if(x == 10) { // start new scope
            int y = 20; // known only to this block
            // x and y both known here.
            System.out.println("x and y: " + x + " " + y);
            x = y * 2;
        }
        // y = 100; // Error! y not known here
        // x is still known here.
        System.out.println("x is " + x);
    }
}
```

Scanner

- `public String nextLine()` :- Moves the scanner to the next line and returns the skipped input.
- `public String next()` :- Returns the token before delimiter
- `public byte nextByte()` :- Scans next token as byte value
- `public short nextShort()` :- Scans next token as short value
- `public int nextInt()` :- Scans next token as integer value
- `public long nextLong()` :- Scans next token as long value
- `public float nextFloat()` :- Scans next token as float value
- `public double nextDouble()` :- Scans next token as double value
- `void close` :- Scanner is closed

next(): Read One word stops at Space, tab, or newline

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your full name: ");
        String name = sc.next();

        System.out.println("Name = " + name);

        sc.close();
    }
}
```

Input: Ashok Rajan

Output: Name = Ashok

nextLine(): Read the entire line

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your full name: ");
String a = sc.nextLine();
System.out.println(a);
```

Input: Ashok Rajan R
Output: Ashok Rajan R

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Read byte
        System.out.print("Enter a byte value: ");
        byte b = sc.nextByte();

        // Read short
        System.out.print("Enter a short value: ");
        short s = sc.nextShort();

        // Read int
        System.out.print("Enter an int value: ");
        int i = sc.nextInt();

        // Read long
        System.out.print("Enter a long value: ");
        long l = sc.nextLong();

        // Read float
        System.out.print("Enter a float value: ");
        float f = sc.nextFloat();

        // Read double
        System.out.print("Enter a double value: ");
        double d = sc.nextDouble();

        System.out.println("\n----- Values Entered -----");
        System.out.println("Byte   : " + b);
        System.out.println("Short  : " + s);
        System.out.println("Int    : " + i);
        System.out.println("Long   : " + l);
        System.out.println("Float  : " + f);
        System.out.println("Double : " + d);

        sc.close();
    }
}
```

```
}  
}
```

Enter a byte value: 100
Enter a short value: 20000
Enter an int value: 500000
Enter a long value: 9876543210
Enter a float value: 12.5
Enter a double value: 45.6789

----- Values Entered -----

Byte : 100
Short : 20000
Int : 500000
Long : 9876543210
Float : 12.5
Double : 45.6789

Non-Primitive (Reference) Data Types

Non-primitive data types are derived from primitive types or created by programmers.

One-Dimensional Arrays

```
public class Main {  
    public static void main(String[] args) {  
  
        int[] arr = {10, 20, 30, 40, 50};  
  
        System.out.println("Array Elements:");  
  
        for (int i = 0; i < arr.length; i++) {  
            System.out.println(arr[i]);  
        }  
    }  
}
```

```

class Testarray{
public static void main(String args[]){
    int a[]=new int[5];//declaration and instantiation

    a[0]=10;//initialization
    a[1]=20;
    a[2]=70;
    a[3]=40;
    a[4]=50;

    //traversing array
    for(int i=0;i<a.length;i++)//length is the of array
        System.out.println(a[i]);
}
}

```

Declaration	Valid?
int[] a;	✓ Recommended
int a[];	✓
int[] a, b;	✓ Both arrays
int a[], b[];	✓ Both arrays
int a[], b;	✓ a array, b int
int a[10];	❑ Invalid in Java

Multidimensional Arrays

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
    }
}

```

```

int[][] a = new int[3][3];

// Input
System.out.println("Enter 9 elements:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        a[i][j] = sc.nextInt();
    }
}

// Display
System.out.println("\nMatrix:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.print(a[i][j] + " ");
    }
    System.out.println();
}

sc.close();
}

```

Input

10 20 30

40 50 60

70 80 90

Output

Matrix:

10 20 30

40 50 60

70 80 90

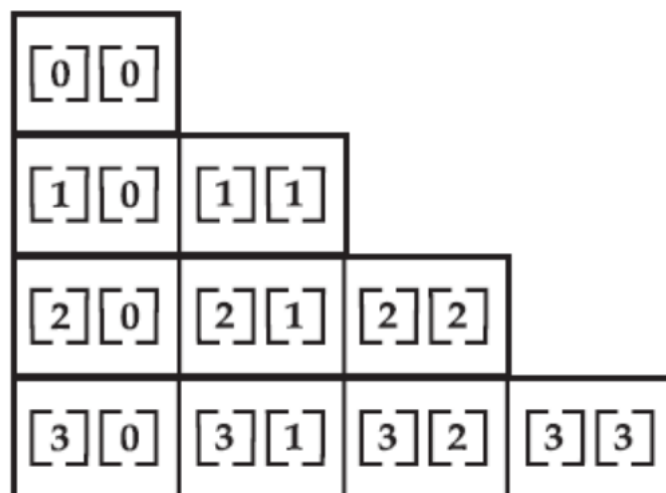
Declaration	Valid?
int[][] a;	✓ Recommended

<code>int a[][];</code>	✓
<code>int[] a[];</code>	✓
<code>int[][] a, b;</code>	✓ Both 2D arrays
<code>int[] a[], b;</code>	✓ a 2D array, b 1D array
<code>int[][] a = new int[3][4];</code>	✓
<code>int a[][] = new int[3][4];</code>	✓

Jagged Array in Java

A Jagged Array (also called a **Ragged Array**) is a 2D array in which each row can have a different number of columns.

- Creating odd number of columns in a 2D array, it is known as a jagged array.
- In other words, it is an array of arrays with **different number of columns**.



```
public class Main {
    public static void main(String[] args) {

        int[][] a = {
            {10, 20, 30},
            {40, 50, 60, 70, 80},
            {90, 100}
        }
    }
}
```

```

};

for (int i = 0; i < a.length; i++) {
    for (int j = 0; j < a[i].length; j++) {
        System.out.print(a[i][j] + " ");
    }
    System.out.println();
}
}
}
}

```

```

10 20 30
40 50 60 70 80
90 100

```

Creating Jagged Array Using new

```

public class Main {
    public static void main(String[] args) {

        int[][] a = new int[3][];

        a[0] = new int[2];
        a[1] = new int[4];
        a[2] = new int[3];

        a[0][0] = 10;
        a[0][1] = 20;

        a[1][0] = 30;
        a[1][1] = 40;
        a[1][2] = 50;
        a[1][3] = 60;

        a[2][0] = 70;
        a[2][1] = 80;
        a[2][2] = 90;

        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[i].length; j++) {
                System.out.print(a[i][j] + " ");
            }
            System.out.println();
        }
    }
}

```

```

    }
}

10 20
30 40 50 60
70 80 90

```

3D Array

```

// Demonstrate a three-dimensional array.
class ThreeDMatrix {
public static void main(String args[]) {
    int threeD[][][] = new int[3][4][5];
    int i, j, k;
    for(i=0; i<3; i++)
        for(j=0; j<4; j++)
            for(k=0; k<5; k++)
                threeD[i][j][k] = i * j * k;

    for(i=0; i<3; i++) {
        for(j=0; j<4; j++) {
            for(k=0; k<5; k++)
                System.out.print(threeD[i][j][k] + " ");
            System.out.println();
        }
        System.out.println();
    }
}
}

```

```

output:
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

0 0 0 0 0
0 1 2 3 4
0 2 4 6 8
0 3 6 9 12

0 0 0 0 0
0 2 4 6 8
0 4 8 12 16
0 6 12 18 24

```

OPERATORS

Operators: Arithmetic Operators, The Bitwise Operators, Relational Operators, Boolean Logic operators, The assignment operator, The ? Operator, Operator Precedence, Using Parentheses.

Arithmetic operators

Operator	Result
+	Addition
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
--	Decrement

// Demonstrate the basic arithmetic operators.

```

class BasicMath {
    public static void main(String args[]) {

        // Arithmetic using integers
        System.out.println("Integer Arithmetic");

        int a = 1 + 1;    // Addition
        int b = a * 3;    // Multiplication
        int c = b / 4;    // Division
        int d = c - a;    // Subtraction
        int e = -d;       // Unary Minus

        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
        System.out.println("d = " + d);
        System.out.println("e = " + e);

        // Arithmetic using doubles
        System.out.println("\nFloating Point Arithmetic");

        double da = 1 + 1;
    }
}

```

```

double db = da * 3;
double dc = db / 4;
double dd = dc - da;
double de = -dd;

System.out.println("da = " + da);
System.out.println("db = " + db);
System.out.println("dc = " + dc);
System.out.println("dd = " + dd);
System.out.println("de = " + de);
}
}

```

Integer Arithmetic

```

a = 2
b = 6
c = 1
d = -1
e = 1

```

Floating Point Arithmetic

```

da = 2.0
db = 6.0
dc = 1.5
dd = -0.5
de = 0.5

```

```

public class IncOp
{
    public static void main(String args[])
    {
        int x=1;
        int r = --x + x++ + ++x + x--;
        System.out.println(x + " " + r);
    }
}

```

1 4

```

public class Test1 {
    public static void main(String args[]) {

        int x = 5;
    }
}

```

```
int r = x++ + ++x;

System.out.println("x = " + x);
System.out.println("r = " + r);
}
}
```

```
public class Test4 {
    public static void main(String args[]) {

        int x = 5;
        int r = ++x + ++x + ++x;

        System.out.println(x + " " + r);
    }
}
```

```
public class Test6 {
    public static void main(String args[]) {

        int x = 5;
        int r = x-- + --x;

        System.out.println(x + " " + r);
    }
}
```

```
public class Test9 {
    public static void main(String args[]) {

        int x = 5;
        int r = x++ + ++x + x++;

        System.out.println(x + " " + r);
    }
}
```

```
public class Main
{
    public static void main(String[] args) {
        int x = 10;
```

```
int a = x++;  
int b = ++x;  
int c = x--;  
int d = --x;  
int r = a + b + c + d;  
System.out.println(x + " " + r);  
}  
}
```

10 44

```
//Guess the output  
public class Test {  
    public static void main(String[] args )  
    {  
        int a = 10;  
        int b = ++(++a);  
        System.out.println(b);  
    }  
}
```

Compilation Error

```
//final variable  
public class Test {  
    public static void main(String[] args )  
    {  
        final int a = 10;  
        int  
        b = ++a;  
        System.out.println(b);  
    }  
}
```

Compilation Error

```
public class Main {  
    public static void main(String[] args) {  
        boolean b = false;  
        b++;  
        System.out.println(b);  
    }  
}
```

Compilation Error

The Bitwise Operators

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

~ (Bitwise Unary NOT)

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5;  
        System.out.println("a = " + a);  
        System.out.println("~a = " + (~a));  
    }  
}
```

a = 5
~a = -6

& (Bitwise AND)

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 3;
```

```
        System.out.println(a & b);
    }
}
```

1

| (Bitwise OR)

```
public class Main {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;

        System.out.println(a | b);
    }
}
```

7

^ (Bitwise XOR)

```
public class Main {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;

        System.out.println(a ^ b);
    }
}
```

6

>> (Signed Right Shift)

Shifts bits to the right while preserving the sign bit.

- If the number is positive, 0s are filled on the left.
- If the number is negative, 1s are filled on the left to preserve the negative sign.

```
// Positive Number
```

```

public class Main {
    public static void main(String[] args) {
        int a = 40;

        System.out.println("a = " + a);
        System.out.println("Binary: " + Integer.toBinaryString(a));

        int b = a >> 1;

        System.out.println("a >> 1 = " + b);
        System.out.println("Binary: " + Integer.toBinaryString(b));
    }
}

```

a = 40
Binary: 101000
a >> 1 = 20
Binary: 10100

```

//Negative Number
public class Main {
    public static void main(String[] args) {
        int a = -40;

        System.out.println("a = " + a);
        System.out.println("Binary = " + Integer.toBinaryString(a));

        int b = a >> 1;

        System.out.println("a >> 1 = " + b);
        System.out.println("Binary = " + Integer.toBinaryString(b));
    }
}

```

a = -40
Binary = 1111111111111111111111111111011000
a >> 1 = -20
Binary = 111111111111111111111111111101100

```

//Divide by 2
public class Main {
    public static void main(String[] args) {

        int x = 64;

```

```

        System.out.println("64 >> 1 = " + (64 >> 1));
        System.out.println("64 >> 2 = " + (64 >> 2));
        System.out.println("64 >> 3 = " + (64 >> 3));
        System.out.println("64 >> 4 = " + (64 >> 4));
    }
}

```

```

64 >> 1 = 32
64 >> 2 = 16
64 >> 3 = 8
64 >> 4 = 4

```

Expression	Binary Operation	Result
40 >> 1	00101000 → 00010100	20
40 >> 2	00101000 → 00001010	10
64 >> 3	01000000 → 00001000	8
-40 >> 1	Sign bit (1) preserved	-20
100 >> 1	Divide by 2	50

>>> (Unsigned Right Shift)

The >>> (Unsigned Right Shift) operator shifts all bits to the right **without preserving the sign bit**.

Unlike >>, it **always fills the leftmost bits with 0s**, regardless of whether the number is positive or negative.

```

//Positive Number
public class Main {
    public static void main(String[] args) {

        int a = 40;

        System.out.println("a = " + a);
        System.out.println("Binary = " + Integer.toBinaryString(a));

        int b = a >>> 1;

        System.out.println("a >>> 1 = " + b);
        System.out.println("Binary = " + Integer.toBinaryString(b));
    }
}

```

```
a = 40
Binary = 101000
a >>> 1 = 20
Binary = 10100
```

```
//Negative number
public class Main {
    public static void main(String[] args) {

        int a = -40;

        System.out.println("a = " + a);
        System.out.println("Binary = " + Integer.toBinaryString(a));

        int b = a >>> 1;

        System.out.println("a >>> 1 = " + b);
        System.out.println("Binary = " + Integer.toBinaryString(b));
    }
}

a = -40
Binary = 1111111111111111111111111111011000
a >>> 1 = 2147483628
Binary = 111111111111111111111111111101100
```

Expression	Result
40 >> 1	20
40 >>> 1	20
-40 >> 1	-20
-40 >>> 1	2147483628
-1 >>> 31	1

<< (Left Shift)

Moves bits to the left

- The **leftmost bits are discarded**.
- The **rightmost positions are filled with 0s**.
- Shifting left by **1** is generally equivalent to multiplying by **2** (provided no overflow occurs).

```

public class Main {
    public static void main(String[] args) {

        int x = 5;

        System.out.println("5 << 1 = " + (5 << 1));
        System.out.println("5 << 2 = " + (5 << 2));
        System.out.println("5 << 3 = " + (5 << 3));
        System.out.println("5 << 4 = " + (5 << 4));
    }
}

5 << 1 = 10
5 << 2 = 20
5 << 3 = 40
5 << 4 = 80

```

&= (Bitwise AND Assignment)

```

public class Main {
    public static void main(String[] args) {
        int a = 5;
        a &= 3;
        System.out.println(a);
    }
}

1

```

|= (Bitwise OR Assignment)

```

public class Main {
    public static void main(String[] args) {
        int a = 5;
        a |= 3;
        System.out.println(a);
    }
}

7

```

^= (Bitwise XOR Assignment)

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5;  
        a ^= 3;  
        System.out.println(a);  
    }  
}  
  
6
```

>>= (Right Shift Assignment)

```
public class Main {  
    public static void main(String[] args) {  
        int a = 20;  
        a >>= 2;  
        System.out.println(a);  
    }  
}  
  
5
```

>>>= (Unsigned Right Shift Assignment)

```
public class Main {  
    public static void main(String[] args) {  
        int a = -20;  
        a >>>= 2;  
        System.out.println(a);  
    }  
}  
  
1073741819
```

<<= (Left Shift Assignment)

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5;  
        a <<= 2;  
        System.out.println(a);  
    }  
}
```

```
}
```

20

Operator	Example	Output
~	~5	-6
&	5 & 3	1
	5 3	7
^	5 ^ 3	6
>>	20 >> 2	5
>>>	-20 >>> 2	1073741819
<<	5 << 2	20
&=	a &= 3	1
=	a = 3	7
^=	a ^= 3	6
>>=	a >>= 2	5
>>>=	a >>>= 2	1073741819
<<=	a <<= 2	20

Relational Operators

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

```
public class Main {  
    public static void main(String[] args) {  
  
        int a = 10;  
        int b = 20;  
        System.out.println("==== Relational Operators ====");  
        System.out.println("a = " + a);  
    }  
}
```

```

System.out.println("b = " + b);

// Equal to
System.out.println("\na == b : " + (a == b));

// Not equal to
System.out.println("a != b : " + (a != b));

// Greater than
System.out.println("a > b : " + (a > b));

// Less than
System.out.println("a < b : " + (a < b));

// Greater than or equal to
System.out.println("a >= b : " + (a >= b));

// Less than or equal to
System.out.println("a <= b : " + (a <= b));

// Another example where values are equal
System.out.println("\n===== When a and b are Equal =====");

a = 20;
b = 20;

System.out.println("a = " + a);
System.out.println("b = " + b);

System.out.println("\na == b : " + (a == b));
System.out.println("a != b : " + (a != b));
System.out.println("a > b : " + (a > b));
System.out.println("a < b : " + (a < b));
System.out.println("a >= b : " + (a >= b));
System.out.println("a <= b : " + (a <= b));
    }
}

```

===== Relational Operators =====

a = 10

b = 20

a == b : false

a != b : true

```
a > b : false
a < b : true
a >= b : false
a <= b : true
```

===== When a and b are Equal =====

```
a = 20
b = 20
```

```
a == b : true
a != b : false
a > b : false
a < b : false
a >= b : true
a <= b : true
```

Operator	Meaning	Example	Result
==	Equal to	10 == 20	false
!=	Not equal to	10 != 20	true
>	Greater than	10 > 20	false
<	Less than	10 < 20	true
>=	Greater than or equal to	20 >= 20	true
<=	Less than or equal to	20 <= 20	true

Boolean Logical Operators

Operator	Result
&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

```
public class Main {
    public static void main(String[] args) {

        boolean a = true;
        boolean b = false;

        System.out.println("==== Boolean Logical Operators =====");

        // 1. Logical AND (&)
        System.out.println("a & b = " + (a & b));

        // 2. Logical OR (|)
        System.out.println("a | b = " + (a | b));

        // 3. Logical XOR (^)
        System.out.println("a ^ b = " + (a ^ b));

        // 4. Short-Circuit AND (&&)
        int x = 10;
        System.out.println("\nShort-Circuit AND (&&)");
        System.out.println(false && (++x > 10));
        System.out.println("x = " + x);

        // 5. Short-Circuit OR (||)
        x = 10;
        System.out.println("\nShort-Circuit OR (||)");
        System.out.println(true || (++x > 10));
        System.out.println("x = " + x);

        // 6. Logical NOT (!)
        System.out.println("\n!a = " + (!a));

        // 7. AND Assignment (&=)
        boolean c = true;
        c &= false;
        System.out.println("\nc &= false -> " + c);

        // 8. OR Assignment (|=)
        c = false;
        c |= true;
        System.out.println("c |= true -> " + c);

        // 9. XOR Assignment (^=)
```

```

c = true;
c ^= true;
System.out.println("c ^= true -> " + c);

// 10. Equal To (==)
int n1 = 10;
int n2 = 10;
System.out.println("\nn1 == n2 : " + (n1 == n2));

// 11. Not Equal To (!=)
n2 = 20;
System.out.println("n1 != n2 : " + (n1 != n2));

// 12. Ternary Operator (?:)
int max = (n1 > n2) ? n1 : n2;
System.out.println("\nMaximum = " + max);

// Difference between & and &&
System.out.println("\n===== Difference between & and && =====");

x = 10;
System.out.println(false & (++x > 10));
System.out.println("x = " + x);

x = 10;
System.out.println(false && (++x > 10));
System.out.println("x = " + x);

// Difference between | and ||
System.out.println("\n===== Difference between | and || =====");

x = 10;
System.out.println(true | (++x > 10));
System.out.println("x = " + x);

x = 10;
System.out.println(true || (++x > 10));
System.out.println("x = " + x);
}
}

===== Boolean Logical Operators =====
a & b = false
a | b = true

```

`a ^ b = true`

Short-Circuit AND (&&)

`false`

`x = 10`

Short-Circuit OR (||)

`true`

`x = 10`

`!a = false`

`c &= false -> false`

`c |= true -> true`

`c ^= true -> false`

`n1 == n2 : true`

`n1 != n2 : true`

`Maximum = 20`

===== Difference between & and && =====

`false`

`x = 11`

`false`

`x = 10`

===== Difference between | and || =====

`true`

`x = 11`

`true`

`x = 10`

The Assignment Operator

```
public class Main {  
    public static void main(String[] args) {  
  
        int a;  
        // Simple Assignment  
        a = 10;  
        System.out.println("a : " + a);  
        // +=  
        a = 10;  
    }  
}
```

```
a += 5;  
System.out.println("+= : " + a);
```

```
// -=  
a = 10;  
a -= 5;  
System.out.println("-= : " + a);
```

```
// *=  
a = 10;  
a *= 5;  
System.out.println("*= : " + a);
```

```
// /=  
a = 10;  
a /= 5;  
System.out.println("/= : " + a);
```

```
// %=  
a = 10;  
a %= 3;  
System.out.println("%= : " + a);
```

```
// &=  
a = 10;  
a &= 6;  
System.out.println("&= : " + a);
```

```
// |=  
a = 10;  
a |= 6;  
System.out.println("|= : " + a);
```

```
// ^=  
a = 10;  
a ^= 6;  
System.out.println("^= : " + a);
```

```
// <<=  
a = 10;  
a <<= 2;  
System.out.println("<<= : " + a);
```

```
// >>=
```

```

    a = 10;
    a >>= 2;
    System.out.println(">>= : " + a);

    // >>>=
    a = -10;
    a >>>= 2;
    System.out.println(">>>=: " + a);
}
}

= : 10
+= : 15
-= : 5
*= : 50
/= : 2
%= : 1
&= : 2
|= : 14
^= : 12
<<= : 40
>>= : 2
>>>=: 1073741821

```

Operator	Example	Equivalent	Result (a=10)
=	a = 10	Assign	10
+=	a += 5	a = a + 5	15
-=	a -= 5	a = a - 5	5
*=	a *= 5	a = a * 5	50
/=	a /= 5	a = a / 5	2
%=	a %= 3	a = a % 3	1

The ? Operator

```

//Find Maximum of Two Numbers
public class Main {
    public static void main(String[] args) {

        int a = 10;
        int b = 20;
        int max = (a > b) ? a : b;
        System.out.println("Maximum = " + max);
    }
}

```

```
}
```

Maximum = 20

Nested Ternary Operator

```
//Largest of Three Numbers
public class Main {
    public static void main(String[] args) {

        int a = 15, b = 20, c = 10;

        int largest = (a > b)? ((a > c) ? a : c): ((b > c) ? b : c);

        System.out.println("Largest = " + largest);
    }
}
```

Largest = 20

Expression	Meaning	Result
(10 > 20) ? 10 : 20	Maximum	20
(10 < 20) ? 10 : 20	Minimum	10
(15 % 2 == 0) ? "Even" : "Odd"	Even/Odd	"Odd"
(75 >= 50) ? "Pass" : "Fail"	Pass/Fail	"Pass"
(-8 >= 0) ? "Positive" : "Negative"	Sign Check	"Negative"

Operator Precedence

Precedence	Operator	Type	Associativity
15	() [] .	Parentheses Array subscript Member selection	Left to Right
14	++ --	Unary post-increment Unary post-decrement	Right to left
13	++ -- + - ! ~ (<i>type</i>)	Unary pre-increment Unary pre-decrement Unary plus Unary minus Unary logical negation Unary bitwise complement Unary type cast	Right to left
12	* / %	Multiplication Division Modulus	Left to right
11	+ -	Addition Subtraction	Left to right
10	<< >> >>>	Bitwise left shift Bitwise right shift with sign extension Bitwise right shift with zero extension	Left to right

9	<	Relational less than	Left to right
	<=	Relational less than or equal	
	>	Relational greater than	
	>=	Relational greater than or equal	
	instanceof	Type comparison (objects only)	
8	==	Relational is equal to	Left to right
	!=	Relational is not equal to	
7	&	Bitwise AND	Left to right
6	^	Bitwise exclusive OR	Left to right
5		Bitwise inclusive OR	Left to right
4	&&	Logical AND	Left to right
3		Logical OR	Left to right
2	? :	Ternary conditional	Right to left
1	=	Assignment	Right to left
	+=	Addition assignment	
	-=	Subtraction assignment	
	*=	Multiplication assignment	
	/=	Division assignment	
	%=	Modulus assignment	

```

public class Main {
    public static void main(String[] args) {

        System.out.println("===== OPERATOR PRECEDENCE
=====\\n");

        // 1. Multiplication has higher precedence than Addition
        int result1 = 10 + 20 * 3;
        System.out.println("1. 10 + 20 * 3 = " + result1);

        // 2. Parentheses override precedence
        int result2 = (10 + 20) * 3;
        System.out.println("2. (10 + 20) * 3 = " + result2);

        // 3. Multiple arithmetic operators
        int result3 = 20 - 10 * 2 + 5;
        System.out.println("3. 20 - 10 * 2 + 5 = " + result3);

        // 4. Relational and Logical Operators
    }
}

```

```

boolean result4 = 10 > 5 && 20 > 30;
System.out.println("4. 10 > 5 && 20 > 30 = " + result4);

// 5. Assignment has lowest precedence
int a;
a = 10 + 5 * 2;
System.out.println("5. a = 10 + 5 * 2 = " + a);

// 6. Mixed expression
int result5 = 10 + 5 * 2 > 15 ? 100 : 200;
System.out.println("6. 10 + 5 * 2 > 15 ? 100 : 200 = " + result5);

System.out.println("\n===== ASSOCIATIVITY =====\n");

// 7. Left-to-Right Associativity (-)
int result6 = 20 - 10 - 5;
System.out.println("7. 20 - 10 - 5 = " + result6);

// 8. Left-to-Right Associativity (/)
int result7 = 100 / 10 / 2;
System.out.println("8. 100 / 10 / 2 = " + result7);

// 9. Left-to-Right Associativity (%)
int result8 = 20 % 7 % 3;
System.out.println("9. 20 % 7 % 3 = " + result8);

// 10. Right-to-Left Associativity (Assignment)
int x, y, z;
x = y = z = 100;
System.out.println("10. x = y = z = 100");
System.out.println("    x = " + x);
System.out.println("    y = " + y);
System.out.println("    z = " + z);

// 11. Right-to-Left Associativity (Unary)
int p = 10;
int q = -(-p);
System.out.println("11. -(-10) = " + q);

// 12. Ternary Operator Associativity
int r = false ? 10 : true ? 20 : 30;
System.out.println("12. false ? 10 : true ? 20 : 30 = " + r);

System.out.println("\n===== SUMMARY =====");

```

```

        System.out.println("Multiplication (*) has higher precedence than
Addition (+).");
        System.out.println("Parentheses () have the highest precedence.");
        System.out.println("Assignment (=) has one of the lowest
precedences.");
        System.out.println("Arithmetic operators (*, /, %) are evaluated before
(+, -).");
        System.out.println("Relational operators are evaluated before Logical
operators.");
        System.out.println("Left-to-Right Associativity: +, -, *, /, %, <, >, ==,
&&, ||");
        System.out.println("Right-to-Left Associativity: =, +=, -=, *=, /=, %=, ?:,
Unary operators");
    }
}

```

===== OPERATOR PRECEDENCE =====

1. $10 + 20 * 3 = 70$
2. $(10 + 20) * 3 = 90$
3. $20 - 10 * 2 + 5 = 5$
4. $10 > 5 \ \&\& \ 20 > 30 = \text{false}$
5. $a = 10 + 5 * 2 = 20$
6. $10 + 5 * 2 > 15 \ ? \ 100 : 200 = 100$

===== ASSOCIATIVITY =====

7. $20 - 10 - 5 = 5$
8. $100 / 10 / 2 = 5$
9. $20 \% 7 \% 3 = 2$
10. $x = y = z = 100$
 $x = 100$
 $y = 100$
 $z = 100$
11. $-(-10) = 10$
12. $\text{false} \ ? \ 10 : \text{true} \ ? \ 20 : 30 = 20$

===== SUMMARY =====

Multiplication (*) has higher precedence than Addition (+).
 Parentheses () have the highest precedence.
 Assignment (=) has one of the lowest precedences.
 Arithmetic operators (*, /, %) are evaluated before (+, -).
 Relational operators are evaluated before Logical operators.

Left-to-Right Associativity: +, -, *, /, %, <, >, ==, &&, ||

Right-to-Left Associativity: =, +=, -=, *=, /=, %=, ?:, Unary operators

```
public class Main {
    public static void main(String[] args) {

        int a, b;

        System.out.println("===== PRE-INCREMENT (++)
=====");
        a = 10;
        b = ++a;
        System.out.println("a = 10");
        System.out.println("b = ++a");
        System.out.println("a = " + a);
        System.out.println("b = " + b);

        System.out.println("\n===== POST-INCREMENT (++)
=====");
        a = 10;
        b = a++;
        System.out.println("a = 10");
        System.out.println("b = a++");
        System.out.println("a = " + a);
        System.out.println("b = " + b);

        System.out.println("\n===== PRE-DECREMENT (--
=====");
        a = 10;
        b = --a;
        System.out.println("a = 10");
        System.out.println("b = --a");
        System.out.println("a = " + a);
        System.out.println("b = " + b);

        System.out.println("\n===== POST-DECREMENT (--
=====");
        a = 10;
        b = a--;
        System.out.println("a = 10");
        System.out.println("b = a--");
        System.out.println("a = " + a);
        System.out.println("b = " + b);
```

```
System.out.println("\n===== COMBINATION EXAMPLES  
=====");
```

```
// Example 1  
a = 10;  
b = ++a + a++;  
System.out.println("\nExample 1");  
System.out.println("a = 10");  
System.out.println("b = ++a + a++");  
System.out.println("a = " + a);  
System.out.println("b = " + b);
```

```
// Example 2  
a = 10;  
b = a++ + ++a;  
System.out.println("\nExample 2");  
System.out.println("a = 10");  
System.out.println("b = a++ + ++a");  
System.out.println("a = " + a);  
System.out.println("b = " + b);
```

```
// Example 3  
a = 10;  
b = --a + a--;  
System.out.println("\nExample 3");  
System.out.println("a = 10");  
System.out.println("b = --a + a--");  
System.out.println("a = " + a);  
System.out.println("b = " + b);
```

```
// Example 4  
a = 10;  
b = a-- + --a;  
System.out.println("\nExample 4");  
System.out.println("a = 10");  
System.out.println("b = a-- + --a");  
System.out.println("a = " + a);  
System.out.println("b = " + b);
```

```
// Example 5  
a = 10;  
b = ++a + --a;  
System.out.println("\nExample 5");  
System.out.println("a = 10");
```

```
System.out.println("b = ++a + --a");
System.out.println("a = " + a);
System.out.println("b = " + b);
```

```
// Example 6
a = 10;
b = a++ + a--;
System.out.println("\nExample 6");
System.out.println("a = 10");
System.out.println("b = a++ + a--");
System.out.println("a = " + a);
System.out.println("b = " + b);
```

```
// Example 7
a = 10;
b = ++a + --a + a++ + a--;
System.out.println("\nExample 7");
System.out.println("a = 10");
System.out.println("b = ++a + --a + a++ + a--");
System.out.println("a = " + a);
System.out.println("b = " + b);
```

```
System.out.println("\n===== IMPORTANT NOTE
=====");
System.out.println("Java evaluates expressions from LEFT to
RIGHT.");
System.out.println("Each ++ or -- immediately updates the variable.");
System.out.println("Unlike C/C++, Java defines the evaluation order
clearly.");
    }
}
```

===== PRE-INCREMENT (++) =====

```
a = 10
b = ++a
a = 11
b = 11
```

===== POST-INCREMENT (++) =====

```
a = 10
b = a++
a = 11
b = 10
```

===== PRE-DECREMENT (--)=

```
a = 10
b = --a
a = 9
b = 9
```

===== POST-DECREMENT (--)=

```
a = 10
b = a--
a = 9
b = 10
```

===== COMBINATION EXAMPLES =====

Example 1

```
a = 10
b = ++a + a++
a = 12
b = 22
```

Example 2

```
a = 10
b = a++ + ++a
a = 12
b = 22
```

Example 3

```
a = 10
b = --a + a--
a = 8
b = 18
```

Example 4

```
a = 10
b = a-- + --a
a = 8
b = 18
```

Example 5

```
a = 10
b = ++a + --a
a = 10
b = 21
```

Example 6

a = 10

b = a++ + a--

a = 10

b = 21

Example 7

a = 10

b = ++a + --a + a++ + a--

a = 10

b = 42